Technical report: booklet-check

- **Track:** Track 2A, OST challenge: Multilingual Natural Language Inference over Swiss Official Voting Booklets
- **Event:** Online
- **Team:** BookletCheck (Hua-I Yu, solo)
- **Demo:** `make run` in `track_2a/` (Docker, two official example cases); full numbers in `eval/RESULTS.md`

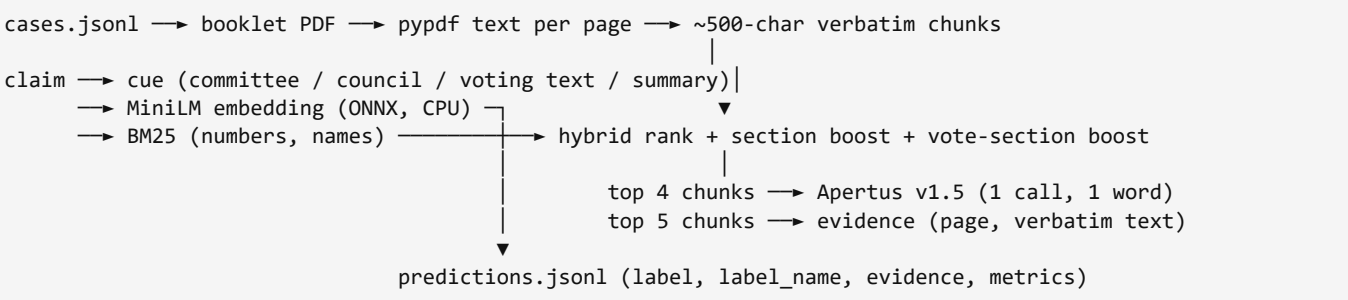
1. Summary

A claim about a Swiss federal vote, in German, French or Italian, has to be labelled entailment, neutral or contradiction against the official voting booklet: either the whole PDF (task A) or one given passage (task B). Task A also asks for evidence with page numbers.

Our system keeps the language model on the one job only it can do. Plain code extracts the PDF text page by page, cuts it into verbatim chunks of about 500 characters, and ranks them with a small multilingual embedding model, BM25 and a structural prior (claims name their source, such as "the committee says" or "according to the voting text", and the booklet sections can be recognised from their page headers). Apertus v1.5 then reads only the top four chunks and answers with a single word. The evidence we return is the five best chunks themselves, with the page they were cut from, so a quote can never be invented and its page is always right.

On the 5 held-out vote dates (389 task A and 389 task B cases), apertus-v1.5-8b with booklet-check reaches macro-F1 0.895 on task A (evidence score 0.804) and 0.949 on task B, with 812 input tokens per task A case. On the same 60-case sample, the whole-booklet baseline scores 0.737 (evidence 0.275) with 38,574 input tokens per case, against 0.917 (evidence 0.900) with 821 for ours. In 18 of those 60 baseline cases the booklet did not fit the context and was cut at 180,000 characters.

2. Architecture



- `src/booklet_check/pdf_index.py`: page extraction (pypdf), chunking along line breaks; every chunk is a contiguous slice of one page, so evidence page numbers are exact.
- `src/booklet_check/retrieve.py`: hybrid ranking. Score = cosine (multilingual MiniLM) + 0.35 x normalised BM25 + 0.08 if the page belongs to the vote named in the case
  - 0.3 if the page is in the section the claim names. Section tags come from page headers ("Argumente Komitee", "Arguments du Conseil fédéral", "Testo in votazione", ...), in all three languages and in both booklet layouts used since 2020.
- `src/booklet_check/pipeline.py`: prompt, label parsing, evidence. Task B uses the same prompt on the given passage; passages longer than 2,600 characters are cut down to their best chunks first.
- `src/booklet_check/llm.py`: thin OpenAI-compatible adapter. It reads `BASE_URL` and `API_KEY` (the names the evaluation injects; the template's `LLM_*` names also work), retries 429 and 5xx with backoff, and records real token counts. With no key it runs a clearly labelled DRY MODE that makes no calls, so `make run` works on a clean checkout.
- Parallelism: retrieval runs under a lock (CPU), model calls run in 4 worker threads.

Cross-lingual claims (for example a German claim about the French booklet) are the reason for the embedding model: BM25 cannot match across languages, while numbers, dates and names still help, which is why both are combined.

3. Use of Apertus

- **Model:** `swiss-ai/apertus-v1.5-8b` (instruct) for every reported run unless the table says 70B; the endpoint's model list is read at start-up when `LLM_NAME` is not set.

- **How it is used:** inference only, one call per case, temperature 0, `max_tokens` 8.
- **Where it runs:** hosted OpenAI-compatible endpoint (Public AI during development; the organisers' proxy during evaluation).
- **Prompt:** a one-line system message ("You verify claims against the official Swiss federal voting booklet. Answer with exactly one word"), then the vote title, the passages with their page numbers, the claim with its language, and short definitions of the three labels (contradiction explicitly includes a different number, date, amount, actor, consequence, position or recommendation). See `pipeline.py`.

Other models. The only other model is `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` (Apache-2.0, 8-bit ONNX, 118 MB) for retrieval. It never sees a label and never produces text; it only orders the chunks. It is downloaded when the Docker image is built (pinned revision, sha256 checked) and read from disk at run time; a CI step runs the image with networking disabled. We also tried `multilingual-e5-small` and `multilingual-e5-base` in the same slot; on the development split neither found clearly more gold passages, so the smallest and fastest model stayed. No other model was used as a judge.

4. Data

- `OSTswiss/MNLIoverSwissVotingBooklets` v1.1 (MIT), 1,488 rows over 20 vote dates, three labels in equal parts, nine language pairs. Cases and gold files are produced with the official `prepare_cases.py`; booklet PDFs come from the Federal Chancellery.
- **Frozen split** (`data/split.json`, `scripts/make_split.py`): by vote date, so that held-out booklets are never seen during development, as the private test set may contain unseen booklets. A hash of each date picks 5 of the 20 dates (389 rows, 778 cases) for the held-out set; the rest (1,099 rows) is for development. The split was committed before the first model call.
- Two observations from the development data shaped the design: neutral claims are claims written for another vote, and the gold passage of a claim is almost always the booklet section that the claim's opening words name (summary, committee, Federal Council, voting text).
- Nothing personal is stored. The repository holds the two official example cases and their booklet; other PDFs are downloaded by `prepare_cases.py --download-booklets`.

5. Evaluation

All scores are computed by the official `evaluate.py` (starter commit `5e69577`) on our prediction files. Every number below is read from `eval/results.json` by `scripts/make_results.py`; `eval/RESULTS.md` has all runs, including development ablations.

The baseline is the same Apertus model with the whole booklet in the prompt (no retrieval), asked for the label, the page and a verbatim quote (`scripts/baseline_full_context.py`). Because of the credit available, the baseline was run on a label-balanced sample of the held-out set; our system was run on the same sample and on the full held-out set.

run	system	model	cases A / B	A macro-F1	A evidence	B macro-F1	input tokens per case A / B	seconds per case A / B	USD
`holdout-ours-8b`	ours	apertus-v1.5-8b	389 / 389	0.895	0.804	0.949	812 / 750	4.5 / 4.6	0.0613
`holdout60-baseline-8b`	baseline	apertus-v1.5-8b	60 / 60	0.737	0.275	0.967	38,574 / 2,096	14.0 / 5.8	0.2451
`holdout60-ours-8b`	ours	apertus-v1.5-8b	60 / 60	0.917	0.900	0.950	821 / 830	10.1 / 6.3	0.0100

6. Limitations

- Evidence is retrieved, not chosen by the model. When the information is repeated in two sections (the summary and the detailed section often say the same thing), our chunk can be right but outside the gold passage, which costs evidence score but not label accuracy.
- The section prior relies on header wording seen in 2020 to 2026 booklets. A new layout would silently turn the boost off (no wrong boost, just none).
- Neutral claims in this dataset come from other votes. If the private set has neutral claims about the same vote that the booklet simply does not settle, the model must recognise that from the passages; we did not hard-code a

similarity cut-off for this reason.

- pypdf text of some older booklets splits words at line ends; the scorer tolerates this, but our BM25 sees two tokens.
- Development numbers come from 8B on a public endpoint with variable latency, so the time per case says little about the official proxy.

7. Reproducibility

```
cd track_2a
make run                                  # Docker, linux/amd64, CPU only; DRY MODE without a key
BASE_URL=... API_KEY=... make run         # real run on data/examples
make run INPUT_DIR=/path/to/data INPUT=cases.jsonl   # any official case file (+ booklets/)
```

Evaluation on the frozen split (Python 3.12, `pip install -r requirements-dev.txt`):

```
python prepare_cases.py --download-booklets --output-dir full   # official starter script
python scripts/split_cases.py --cases full/cases.jsonl --expected full/expected-labels.jsonl \
       --part holdout --out-dir work/holdout
APERTUS_BASE_URL=... APERTUS_API_KEY=... EMBED_DIR=<model dir> \
python scripts/run_eval.py --system ours --split-dir work/holdout --booklet-root full \
       --tag holdout-ours --official-eval <starter>/evaluate.py
python scripts/make_results.py
```

Temperature is 0 and there is no sampling elsewhere, so reruns differ only by endpoint non-determinism. CI (`.github/workflows/ci.yml`) runs the unit tests, builds the image, runs `make run` and checks the output contract, and reruns the image with networking disabled.

8. Next steps

- Let Apertus pick which of the five evidence chunks decides the case (one extra token) and put it first.
- Learn the section prior from the development data instead of header rules.
- A cached claim translation for the few cross-lingual cases where retrieval is unsure.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). Code is Apache-2.0, data we add is CDLA-Permissive-2.0. All Hack Apertus projects are open source.

References

- OST dataset: https://huggingface.co/datasets/OSTswiss/MNLIoverSwissVotingBooklets
- Official starter and evaluator: https://gitlab.com/ifsoftware/hackapertus-starter
- Apertus: https://huggingface.co/swiss-ai
- Reimers and Gurevych (2020), Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation (paraphrase-multilingual-MiniLM).
- Robertson and Zaragoza (2009), The Probabilistic Relevance Framework: BM25 and Beyond.